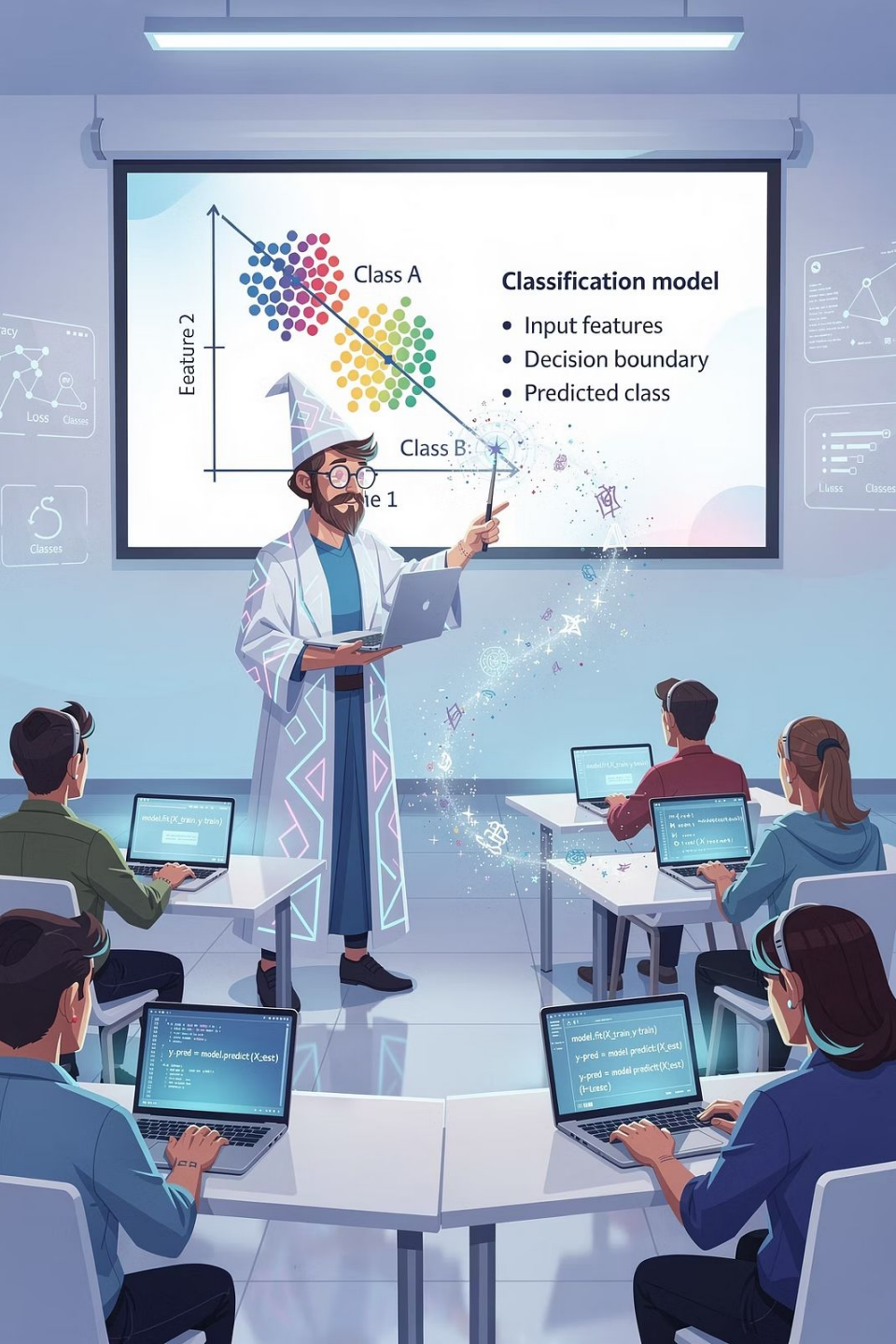




Model Families – Instance-Based



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>



Lazy vs. Eager Learning

The fundamental difference between these two paradigms lies in *when* the model is built and *how* predictions are made.

Eager Learning

Methods such as Naïve Bayes, decision trees, and neural networks build a classification model **before** receiving new data to classify.

- More time during training
- Less time during prediction
- Commits to a single global hypothesis

Accuracy Advantage

Lazy methods effectively use a richer hypothesis space, as they rely on many local linear functions to form an implicit global approximation of the target function.

Lazy Learning

Instance-based methods simply store the training data and **wait** until a test tuple is received for classification.

- Less time during training
- More time during prediction
- Uses multiple local linear functions

❏ Lazy learning sacrifices prediction speed for greater flexibility and accuracy potential, especially in complex decision spaces.

Instance-Based Methods

Instance-based learning is a family of algorithms that store training examples and defer processing until a new instance needs to be classified.

Core Principle

Rather than abstracting data into a model, instance-based methods keep examples in memory and use them directly to make predictions. Classification is based on **similarity** between instances.

k-Nearest Neighbors (k-NN)

Instances are represented as points in a Euclidean space. Classification is based on a majority vote among the k nearest neighbors.

Locally Weighted Regression

Builds local approximations using regression weighted by neighbors. Closer points have greater influence on the prediction.

Case-Based Reasoning

Uses symbolic representations and knowledge-based inference. Retrieves similar cases and adapts their solutions to the new problem.

Each approach offers different trade-offs between computational complexity, interpretability, and accuracy. k-NN is the most popular method due to its simplicity and effectiveness.

The k-Nearest Neighbors Algorithm

Core Concept

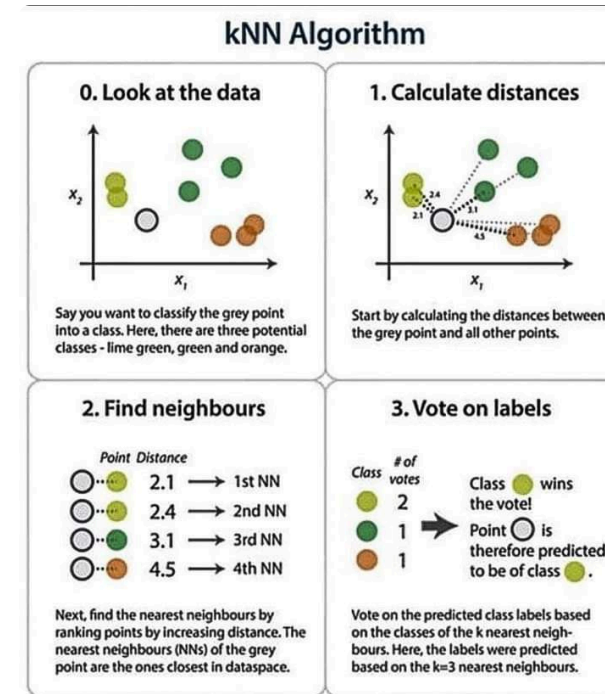
k-NN classifies a new instance based on the classes of its k nearest neighbors in the feature space.

Classification Process

1. **Compute distances:** Measure the distance between the new instance and all training examples
2. **Select neighbors:** Identify the k closest examples
3. **Vote:** Assign the most common class among the k neighbors

Choosing k

The value of k is critical: small values are sensitive to noise, while large values may include points from other classes.



k = 1

Single neighbor: fast but sensitive to outliers

k = 3

Common choice: good balance between bias and variance

k = 5

More robust: reduces the impact of noise

k-NN Visualization

This visualization illustrates how the k-NN algorithm classifies a new point x_q based on its nearest neighbors. The symbols represent different classes: circles (+), squares (-), and triangles (.).



K-Nearest Neighbors – Theory

k-NN classifies a new instance x_q by finding the k training examples closest to it (using a distance metric, typically Euclidean) and assigning the majority class among those neighbors.

Distance Metric

The most common distance metric is Euclidean distance, defined as:

$$d(x_i, x_j) = \sqrt{\sum_{a=1}^D (x_{ia} - x_{ja})^2}$$

Where x_i and x_j are two instances, a iterates over the D features. It's crucial that all features are normalized or standardized to prevent features with larger scales from disproportionately influencing the distance calculation.

Decision Rule

The predicted class $\hat{y}$ for the new instance is determined by a majority vote among the classes of its k nearest neighbors:

$$\hat{y} = \text{mode}\{y(x_1), \dots, y(x_k)\}$$

Where $y(x_m)$ is the class of the m -th nearest neighbor.

Complexity

k-NN is a lazy learning algorithm:

- **Training:** $O(1)$ – simply storing the training data.
- **Prediction:** $O(n \cdot d)$ – where n is the number of training examples and d is the number of features. This involves calculating distances to all training points.

Key Parameters

k (Number of Neighbors)

Controls the bias-variance trade-off. A small k can lead to high variance (sensitive to noise), while a large k can lead to high bias (oversmoothing).

Distance Metric

Euclidean is the default, but other metrics like Manhattan or Minkowski can be used depending on the data characteristics.

Feature Scaling

Mandatory for distance-based algorithms. Use normalization (Min-Max) or standardization (Z-score) to ensure all features contribute equally.

 Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. IEEE Transactions on Information Theory, 13(1), 21–27.
<https://doi.org/10.1109/TIT.1967.1053964>

K-Nearest Neighbors: `cla_knn`

```
# Loading the package
library(daltoolbox)

# Data and levels
iris <- datasets::iris
slevels <- levels(iris$Species)

# Train/test split
set.seed(1)
sr <- sample_random()
sr <- train_test(sr, iris)
iris_train <- sr$train
iris_test <- sr$test

# Training — k=1 (nearest neighbor)
model <- cla_knn("Species", sleveles, k=1)
model <- fit(model, iris_train)
```

```
# Evaluation — training
train_pred <- predict(model, iris_train)
train_eval <- evaluate(model,
  adjust_class_label(iris_train[, "Species"]),
  train_pred)
# accuracy: 1.000

# Evaluation — test
test_pred <- predict(model, iris_test)
test_eval <- evaluate(model,
  adjust_class_label(iris_test[, "Species"]),
  test_pred)
# accuracy: 0.933
```

Step-by-step

- Use `cla_knn("Species", sleveles, k=1)` — main parameter: `k`
- Small `k` (e.g., 1) may overfit noise in training data
- Larger `k` smooths the decision boundary
- Training accuracy: **1.000** | Test accuracy: **0.933** — the gap reflects sensitivity to `k`
- Classifies by majority vote among the `k` nearest neighbors in the feature space

📌 **Key Concept:** KNN is an **instance-based** method — there is no explicit model, only distances. The value of `k` controls the bias-variance trade-off.

Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

